Programming Problem Difficulty Predictor

1. Introduction

Online competitive programming platforms such as Codeforces, CodeChef, and Kattis categorize programming problems into difficulty levels like Easy, Medium, and Hard. These difficulty labels are often assigned based on human judgment and user feedback, which can be subjective and inconsistent. This project aims to automate the difficulty prediction process using Natural Language Processing (NLP) and Machine Learning techniques.

The system predicts both a categorical difficulty level and a numerical difficulty score using only the textual content of programming problems.

---

2. Problem Statement

The objective of this project is to build an intelligent system that can automatically: - Classify programming problems into Easy, Medium, or Hard - Predict a numerical difficulty score

The prediction is based solely on textual information such as: - Problem title - Problem description - Input description - Output description

The system should also provide a simple web interface where users can input a new problem description and obtain predictions.

---

3. Dataset Description

The dataset used in this project consists of programming problems collected in JSON format.

Dataset details: - File: `data/problems.json` - Number of samples: Approximately 4000

Features included: - `title` - `description` - `input_description` - `output_description` - `problem_class` (Easy / Medium / Hard) - `problem_score` (numerical difficulty score) - `url`

The dataset already contains labeled difficulty classes and scores, so no manual labeling was required.

---

4. Data Preprocessing

Before training the models, the textual data is cleaned and prepared to improve prediction performance.

The following preprocessing steps were applied: - Missing values in text fields were handled by replacing them with empty strings. - All text was converted to lowercase to ensure consistency. - Punctuation and unnecessary symbols were removed implicitly during vectorization. - The title,

description, input description, and output description were concatenated into a single text field called `combined_text`.

This combined text representation allows the model to understand the complete context of a programming problem.

---

5. Feature Engineering

After preprocessing, textual data was converted into numerical features using TF-IDF (Term Frequency–Inverse Document Frequency).

TF-IDF Vectorizer: - Captures important keywords related to problem difficulty - Reduces the impact of commonly occurring words - Uses unigrams and bigrams - Limits vocabulary size using `max_features`

The fitted TF-IDF vectorizer was saved and reused during inference to ensure consistent feature representation.

---

6. Models Used

6.1 Classification Model

- Model: Random Forest Classifier
- Target variable: `problem_class`
- Classes: Easy, Medium, Hard

Random Forest was chosen for its robustness and ability to handle high-dimensional sparse features produced by TF-IDF.

6.2 Regression Model

- Model: Random Forest Regressor
- Target variable: `problem_score`

The regression model predicts a continuous difficulty score that complements the categorical difficulty label.

---

7. Experimental Setup

- Data was split into training and testing sets using an 80:20 ratio.
- The same TF-IDF features were used for both classification and regression tasks.
- Models were trained using default hyperparameters with a fixed random state for reproducibility.

Trained models and the TF-IDF vectorizer were saved using Joblib for later use in the web application.

---

8. Results and Evaluation

8.1 Classification Performance

- Accuracy: ~55%
- Confusion Matrix: Generated in the training notebook

The moderate accuracy reflects the subjective nature of difficulty labels, which can vary across platforms and users.

8.2 Regression Performance

- Mean Absolute Error (MAE): ~1.70
- Root Mean Squared Error (RMSE): ~2.04

These results indicate that the model provides reasonable numerical difficulty predictions.

---

9. Web Interface

A web application was developed using Streamlit to allow interactive use of the trained models.

Features of the Web App: - Text input fields for problem title, description, input, and output - Predict button to trigger inference - Displays predicted difficulty class and difficulty score

The application loads pre-trained models and performs real-time predictions without retraining.

---

10. Sample Prediction

Users can paste a new programming problem into the web interface. After clicking the Predict button, the system outputs: - A predicted difficulty class (Easy / Medium / Hard) - A numerical difficulty score

This demonstrates the end-to-end functioning of the NLP and machine learning pipeline.

---

11. Conclusion

This project demonstrates how NLP techniques combined with machine learning can be effectively used to predict the difficulty of programming problems. Although difficulty is inherently subjective, the system provides meaningful predictions and a practical interactive interface.

The project successfully integrates data preprocessing, feature engineering, model training, evaluation, and deployment through a web application.

---

12. Author Details

- Name: Himani Rohaj

- Program: BS-MS (Mathematics and Computing)
- Project Type: NLP + ML + Web Application
- GitHub: https://github.com/z3robyte18